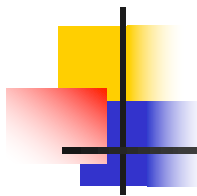


Corso di Crittografia

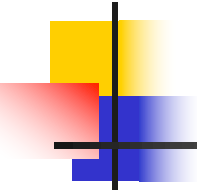
Prof. Dario Catalano

Funzioni Hash



Introduzione

- Una funzione hash e' (generalmente) una funzione che comprime.
- Riceve un input di lunghezza arbitraria e restituisce un output di lunghezza fissata (es. 128,160,256 bit).
- Vi sono molti tipi (diversi) di funzioni hash utili alla crittografia.
- Oggi studieremo tali funzioni.



Resistenza alle Collisioni

Def: Una collisione per $H:D \rightarrow R$ è una coppia x_1, x_2 tale che $H(x_1)=H(x_2)$ ma $x_1 \neq x_2$

Se $|D| > |R|$ le collisioni sono inevitabili.

→ PIGEON HOLE

H è **resistente alle collisioni** se è **computazionalmente** impossibile trovare collisioni.

In crittografia non vogliamo collisioni

Famiglie di funzioni

chiave

- $H: \{0,1\}^k \times D \rightarrow \{0,1\}^n$ famiglia di funzioni
 - Ogni chiave K definisce una mappa

$$h(x) = H_K(x)$$

fissata K abbiamo una specifica istanza

- K è resa pubblica *non segreto*
- Parleremo di resistenza alle collisioni per **famiglie** di funzioni

$$\{0,1\}^{256} \rightarrow \{0,1\}^{160}$$

256

$$\frac{2^{256}}{2^{160}} = 2^{96}$$

Resistenza alle collisioni

- E' sorprendente che tale proprieta' sia possibile per funzioni non limitiamo la dim dell'input.
- Anche se ci limitassimo a input di 256 bit ci avremmo un numero ENORME di input che collidono.
- Principio della Piccionaia: Se 2^{256} piccioni cercano posto in una piccionaia con 2^{160} posti, almeno due piccioni dovranno condividere il posto.
- In realta' la nostra piccionaia e' estremamente sovraffollata. $\rightarrow 2^{96}$ input hanno lo stesso output.

2^{96} input hanno lo stesso output.

Definizione (funzioni Universali o quasi Universali)

■ $H: K \times D \rightarrow R$ funzione hash

$$|D| > |R|$$

COLLISION RESISTANT

$\text{Esp}_H^{\text{cr0}}(A)$

SCEGLIAMO LA COPPIA PRIMA CHE SAPPIAMO L'ESISTENZA DI FUNZIONE

$(x_1, x_2) \leftarrow_R A(); k \leftarrow_R K;$

If $((H_k(x_1) = H_k(x_2)) \text{ and } (x_1 \neq x_2) \text{ and } (x_1, x_2 \in D))$

Return 1

else Return 0

$$\text{Adv}^{\text{cr0}}(A) = \Pr[\text{Esp}_H^{\text{cr0}}(A) = 1]$$

È un numero se Adv è prossimo a 0 PER TUTTI GLI AVVERSARI ESISTENTI

In tal caso il tempo di calcolo dell'avversario non è rilevante, quanto in altre definizioni.

Definizione (funz. Universali Unidirezionali)

■ $H: K \times D \rightarrow R$ funzione hash
Known KEY

$\text{Esp}_H^{\text{cr1-kk}}(A)$
Fissa un input si sceglie la chiave in base alla chiave scelta x_2 memoria
 $(x_1, \text{stato}) \leftarrow_R A(); k \leftarrow_R K; x_2 \leftarrow_R A(k, \text{stato});$
come prima { If $((H_k(x_1) = H_k(x_2)) \text{ and } (x_1 \neq x_2) \text{ and } (x_1, x_2 \in D))$
Return 1
else Return 0

$$\text{Adv}^{\text{cr1-kk}}(A) = \Pr[\text{Esp}_H^{\text{cr1-kk}}(A) = 1]$$

$$y \in R$$

$$S_y = \{ \underset{\text{ELEMENTI DI}}{x \in D} \mid \underset{\text{PER CUI}}{H_K(x) = y} \}$$

l'hash è

$$H_K^{-1}(y) = z \quad z \in S_y$$

$$\text{inverti}(K, y) \rightarrow x \in S_y$$

Si dice unidirezionale se, a partire da x posso calcolare y ma nessun avversario computazionale limitato può calcolare gli altri x che danno y

$$A(\gamma) \rightarrow x_1 \in D$$

$$x_1 \neq x_2$$

$$\Delta(K, \text{stato})$$

$$y \leftarrow H_K(x_1)$$

$$z \leftarrow \text{inverti}(K, y) \rightarrow \text{l'attacker mi dà un altro punto su tutti quelli che danno lo stesso } y$$

$$x_2 \leftarrow z$$

$$\text{IF}(x_1 \neq x_2) \rightarrow \text{Output } (x_1, x_2)$$

* Supponiamo per assurdo che H non sia una funz. universale ma
 $\rightarrow \exists B$ che attacca H come funz. universale
 Allora costruisco A che attacca
 come funz. unidirezionale

Osservazioni

- Chiaramente una funzione Universale Unidirezionale e' anche una funzione universale.

$$\text{Adv}^{\text{cr0}}(B) \leq \text{Adv}^{\text{cr1-kk}}(A)$$

- La dimostrazione di questo fatto e' molto semplice.

Hp: H e' una funzione universale unidirezionale

Ts: H e' una funzione universale.

✱

$B()$
 output (x_1, x_2)

$A()$
 Run $B() \rightarrow (x_1, x_2)$
 $state \leftarrow x_2$
 Output $(x_1, state)$

Codiamo in assurdo

$A(k, state)$
 $x_2 \leftarrow state$
 Output x_2

$$\begin{aligned} \text{Adv}(A) &= \Pr[\text{Esp}^{\text{crs-1-kk}}(A) = 1] = \Pr[\text{Esp}^{\text{cro}}(B) = 1] \\ &= \text{Adv}(B) \end{aligned}$$

30/10/2025

Definizione

(Funzioni Resistenti alle Collisioni)

■ $H: K \times D \rightarrow R$ funzione hash

$|D| \gg |R|$

$\text{Esp}_H^{\text{cr2-kk}}(A)$

$k \leftarrow_R K; (x_1, x_2) \leftarrow_R A(k);$ SCEGLIAMO LA CHIAVE E POI FACCIAMO L'ATTACCO

If $((H_k(x_1) = H_k(x_2)) \text{ and } (x_1 \neq x_2) \text{ and } (x_1, x_2 \in D))$

Return 1

else Return 0

$$\text{Adv}^{\text{cr2-kk}}(A) = \Pr[\text{Esp}_H^{\text{cr2-kk}}(A) = 1]$$

Una funz. res. alle coll. e' anche una funz. universale unidirezionale. SE RESISTE A COLLISIONI È UNIDIREZ. E UNIVERSALE

Hp: H è resistente alle collisioni

TS: H è universale unidirezionale

Supponiamo per assurdo che H non sia universale unidirezionale.

$\leadsto \exists B$ che attacca H come f. U.U.

$B() \rightarrow x_1, \text{ stato}$

$B(K, \text{ stato}) \rightarrow x_2$

$A(K)$

Run $B() \rightarrow (x_1, \text{ stato})$

Run $B(K, \text{ stato}) \rightarrow x_2$

$\exists$ è una collisione
output (x_1, x_2) , ELSE $x_1, x_2 \xleftarrow{R} \mathcal{X} : x_1 \neq x_2$

$$\begin{aligned} \text{Adv}(A) &= \Pr [E_{\text{SP}}^{\text{CR2-KK}}(A) = 1] = \Pr [E_{\text{SP}}^{\text{CR1-KK}}(B) = 1] = \\ &= \text{Adv}(B) \end{aligned}$$

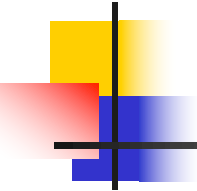
$$\begin{aligned} \text{Adv}(A) &= \Pr [E_{\text{SP}}^{\text{CR2-KK}}(A) = 1] = \Pr [E_{\text{SP}}^{\text{CR1-KK}}(B) = 1] + \text{A CASO} \\ &\geq \text{Adv}(B) \end{aligned}$$

FISSATO K POTREI TROVARE UN ATTACCO PER LA SPECIFICA CHIAVE MA NON PER ALCUNA.

DI FATTO ALCUNI HASH USANO CHIAVI FISSE O NON NE USANO PROPRIO.

SE FISSO UNA CHIAVE POSSO FARE BRUTE FORCING DI AVVERSAIO E TROVARE ADV CHE DA IN OUTPUT SOLO x_1, x_2 A CASO CHE POTREBBE DARE COLLISIONI.

CONVINCIAMOCI CHE SERVONO LE COLLISIONI!



Esempio

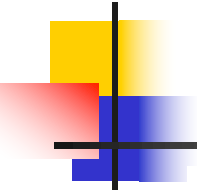
- $H: \{0,1\}^k \times \{0,1\}^{256} \rightarrow \{0,1\}^{128}$

$$H_k(x) = \text{AES}_k(x[1]) \oplus \text{AES}_k(x[2])$$

- Resistente alle collisioni? **No!** SE $H_0(x_L) = H_0(x_R)$
CI SONO PROBLEMI

STANDARD FINO AL 2006

Funzioni hash in pratica: SHA1 (Secure Hash Algorithm 1)



$$\text{SHA1} : \{0, 1\}^{<2^{64}} \rightarrow \{0, 1\}^{160}$$

- Prende in input valori di taglia (praticamente) illimitata e produce output di 160 bit.
- Fu proposta nel 1995
- E' (non più) molto usata in pratica.

COMPRESSIONE MAPPA DA L a ℓ

HASH MAPPA DA VARIABILE A ℓ

La funzione SHA1 – Storia

- SHA1 deriva dalla funzione MD4, proposta da Ron Rivest nel 1990.
- Un'altra funzione (molto usata in pratica) è MD5
- Gli algoritmi SHA1, MD4 e MD5 sono tutti piuttosto simili.
- Idea: concatenare una funzione di compressione che va da $512 + \ell$ bit a ℓ bit. $512 + \ell \rightarrow \ell$
- Per MD4 e MD5 $\ell = 128$
- Per SHA1 $\ell = 160$.

La funzione SHA1 –

Idee di base

- Si compone di diverse procedure:
 - Inizia col modificare l'input M attraverso la procedura SHAPAD → OUTPUT SPECIFICO PER POTER COMPRIMERE
 - Quindi viene iterata la funzione di compressione shf1
- Non descriveremo molto in dettaglio SHA1.
- Ignoreremo del tutto MD4 e MD5.
- Fra qualche lucido vedremo perche'.

SHA1(M) $// |M| < 2^{64}$

$V \leftarrow \text{SHF1}(5A827999 || 6ED9EBA1 || 8F1BBCDC || CA62C1D6, M)$

return V

SHF1(K, M) $// |K| = 128 \text{ e } |M| < 2^{64}$

$y \leftarrow \text{shapad}(M)$ $\rightarrow$ LUNGHEZZA MULTIPLO DI 512

Sia $y \ M_1 || M_2 || \dots || M_n$ ($|M_i| = 512$)

$V \leftarrow 67452301 || EFCDA89 || 98BADCFE || 10325476 || C3D2E1F0$

for $i = 1, \dots, n$

$V \leftarrow \text{shf1}(K, M_i || V) \rightarrow$

return V $512 + 160 = 160$

shapad(M) $// |M| < 2^{64}$

$d \leftarrow (447 - |M|) \bmod 512$

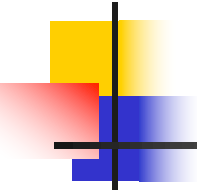
Sia ℓ la rappr. binaria (64 bit) di $|M|$ $\rightarrow$ MAX 2^{64} BIT E LO RAPPRESENTO IN 64 BIT

$y \leftarrow M || 1 || 0^d || \ell$ $// |y|$ multiplo di 512

return y $\uparrow \uparrow$

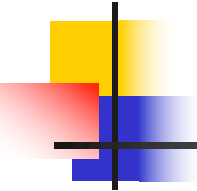
$$|y| = |M| + 1 + d + 64 = 447 + 1 + 64 + t \cdot 512$$

$$191 + d = 447 + t512$$



Applicazioni di funzioni hash

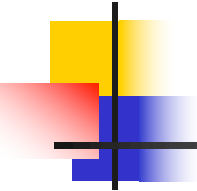
- Ingrediente fondamentale di molte primitive crittografiche
- Utile in molte applicazioni di security
- Utile anche in applicazioni che non hanno a che fare con sicurezza/crittografia



Password Verification

- Client A ha una password pw condivisa col server B
- A può farsi "riconoscere" inviando pw a B attraverso un canale sicuro
 - Es. SSL
- Problema: se il server viene attaccato pw è compromessa

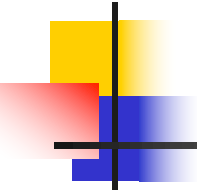
BRUTE FORCE N' SE PÙ FA'



Compare by hash

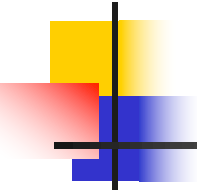
- A hanno due enormi files FA e FB
- A e B vogliono scoprire se i due files coincidono
- La soluzione ovvia può essere molto dispendiosa. $\rightarrow$ bit $\times$ bit non può funzionare.

MEGLIO MANDARE 128 bit



Compare by hash

- A hanno due enormi files FA e FB
- A e B vogliono scoprire se i due files coincidono
- A calcola $h(FA)$ e lo invia a B.
- B verifica che $h(FA)=h(FB)$.



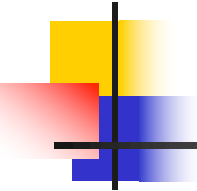
Virus Protection

- Un eseguibile X è disponibile in diversi siti $S_1, S_2, \dots, S_N$.
- Quale di essi è affidabile?

Soluzione:

- Una volta procuratosi $h=H(X)$ (in modo sicuro) si può scaricare X da qualunque sito!

//
CONTROLLI DI INTEGRITÀ



Attacchi alle funzioni hash

- Ci concentriamo sulla proprietà di resistenza alle collisioni (CR2).
- Questa è la proprietà più utile in pratica.
- In particolare mostreremo solo attacchi generici.
- Accenneremo soltanto ad attacchi più specifici.

$$H : D \rightarrow \{0,1\}^L$$

SUPPONIAMO UNA F. HASH
PERFETTA

Definizione

(Funzioni Resistenti alle Collisioni)

■ $H: K \times D \rightarrow R$ funzione hash

$\text{Esp}_H^{\text{cr2-kk}}(A)$

$k \leftarrow_R K; (x_1, x_2) \leftarrow_R A(k);$

If $((H_k(x_1) = H_k(x_2)) \text{ and } (x_1 \neq x_2) \text{ and } (x_1, x_2 \in D))$

Return 1

else Return 0

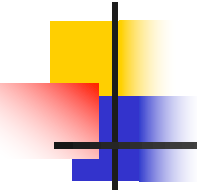
$$\text{Adv}^{\text{cr2-kk}}(A) = \Pr[\text{Esp}_H^{\text{cr2-kk}}(A) = 1]$$

Una funz. res. alle coll. e' anche una funz. universale unidirezionale.

2^L

Forza Bruta

- Scegliamo un input a caso x_1 e calcoliamo $H(x_1)$
- Proviamo tutti i possibili input x_2 fino a quando $H(x_1) = H(x_2)$ ma $x_1 \neq x_2$
- Nel caso di SHA1 tale attacco potrebbe costare 2^{160} passi!
 CASO PEGGIORE. TUTTI GLI OUTPUT
DIVERSI SONO FINITI
- Possiamo far meglio?

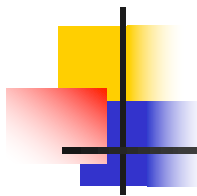


Attacco del compleanno

- Idea: Prendiamo q input a caso e calcoliamo i corrispondenti valori della funzione.
- Se troviamo 2 punti distinti che collidono, abbiamo vinto
- Il paradosso del compleanno sembrerebbe garantire che per trovare una collisione bastano

$$q \approx O\left(\sqrt{|R|}\right)$$

tentativi.



Analisi

Sia $H: \{0,1\}^k \times D \rightarrow \{0,1\}^n$

$$q \approx \sqrt{|D|}$$

$A(K)$

for $i=1, \dots, q$ **do**

$x_i \leftarrow_R D; y_i \leftarrow H_k(x_i);$

if $\exists i, j (i \neq j \wedge y_i = y_j \wedge x_i \neq x_j)$ **return** x_i, x_j

else return FAIL

2^{80}
SCELGO x_i a CASO E LO CIFRO

A vs Attacco del Compleanno

Avversario A

for $i=1,\dots,q$ **do**

$x_i \leftarrow_R D; y_i \leftarrow H_k(x_i);$

if $\exists i,j (i \neq j \wedge y_i = y_j \wedge x_i \neq x_j)$
COLL $\leftarrow$ True

$\Pr[\text{COLL}] = ?$

*y non è distribuito
in maniera uniforme*

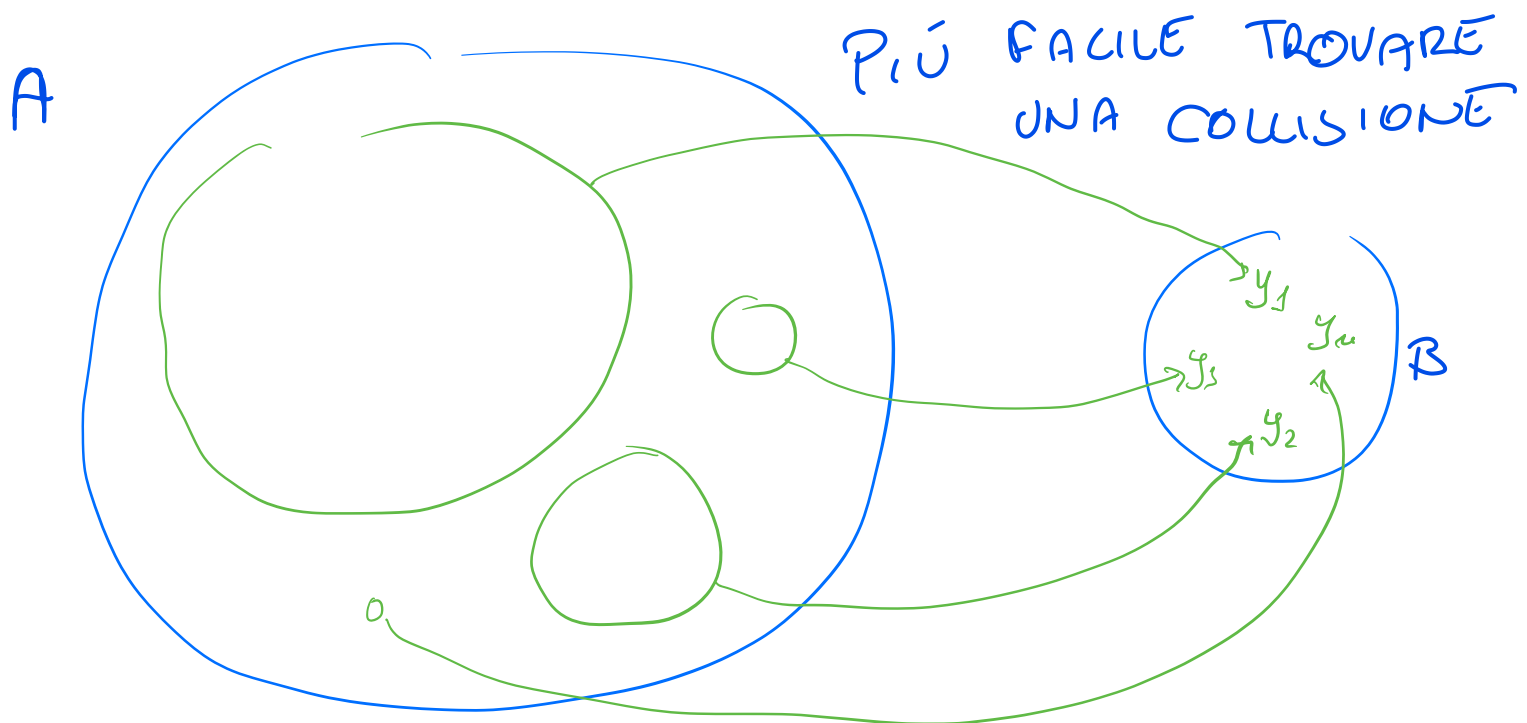
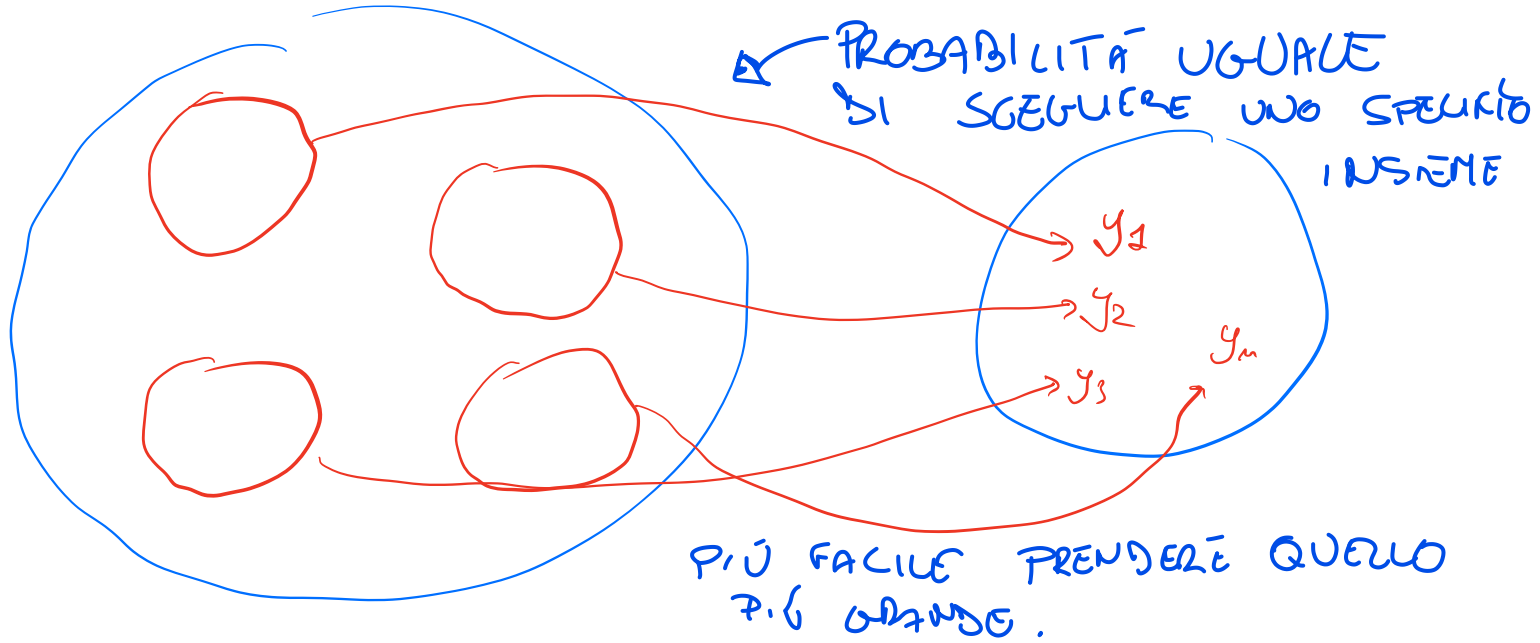
Attacco del Compleanno

for $i=1,\dots,q$ **do**

$y_i \leftarrow_R \{0,1\}^n;$

if $\exists i,j (i \neq j \wedge y_i = y_j)$ **COLL**
 $\leftarrow$ True

$\Pr[\text{COLL}] = q^2 / 2^{n+1}$



Non è importante la dimensione del dominio,
 è importante la dimensione dell'output. Se ho il
 coDominio grande vuol dire che nel dominio
 A ho più palline che portano a elementi
 diversi del coDominio.

COdominio GRANDE È CONDIZIONE NECESSARIA!

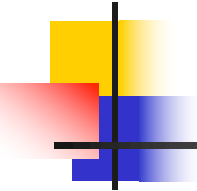
L'OUTPUT HASH È IL DOPPIO RISPETTO AL NUOVO
ATTACCO DEL COMPLEANNO ATTUALMENTE ~~NON~~ FATTIBILE

In particolare

- L'analisi dell'attacco del compleanno, assume che ogni x abbia la stessa probabilità di "produrre" ogni $H_k(y)$
- Questo può non essere vero nel nostro caso.
- Sia $P(y)$ la probabilità che $H_k(x)=y$

$$P(y) = \frac{|H_k^{-1}(y)|}{|D|}$$

NUMERO DI INPUT
DOMINIO



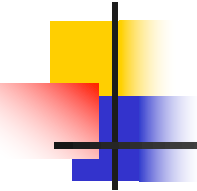
Funzioni Regolari

- Una funzione con la proprietà che

$$\forall y_1, y_2 \quad |H_k^{-1}(y_1)| = |H_k^{-1}(y_2)|$$

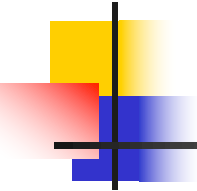
e' detta regolare.

- Per tali funzioni hash possiamo applicare l'analisi dell'attacco del compleanno.
- Cosa possiamo dire per le funzioni non regolari?



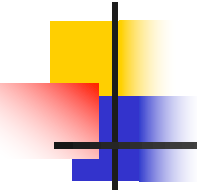
Funzioni non regolari

- Se H non è regolare, l'attacco può solo diventare più efficace.
 - Troviamo collisioni prima
- Per questa ragione una buona funzione hash crittografica dovrebbe essere quanto più regolare possibile.



Conseguenze dell'attacco (caso di SHA1)

- SHA1 utilizza come insieme di arrivo $\{0,1\}^{160}$.
- Assumendo che l'attacco del compleanno sia il migliore possibile, bisognerebbe calcolare SHA1 2^{80} volte per trovare una collisione.
- Tale valore, renderebbe l'operazione non fattibile in pratica.
- Quanto e' ragionevole tale ipotesi?



Cosa succede in pratica

- Abbiamo soltanto accennato alla funzione SHA1.
- Non descriveremo ne' MD4 e nemmeno MD5.
- Esistono ragioni piuttosto forti alla base di tali scelte.



When	Against	Time	Who
1993,1996	md5	2^{16}	[dBBo,Do]
2005	RIPEMD	2^{18}	[JoCaLeJa] [WaFeLaYu] [WaYiYu,WaYaYa] [MHP] [WaFeLaYu,LeWadW,KI]
2004	SHA0	2^{51}	
2005	SHA0	2^{40}	
2005	SHA1	$2^{69}, 2^{63}$	
2009	SHA1	2^{52}	
2005,2006	MD5	1 minute	

STANDARDS → SKILL ISSUE

md5 is the compression function of MD5

SHA0 is an earlier, weaker version of SHA1

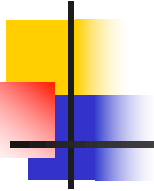
SKIFF!



Attacchi a MD5

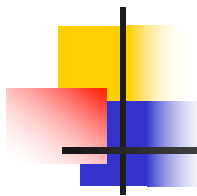
- Attualmente possiamo effettuare i seguenti attacchi
 - Trovare due msg apparentemente casuali che differiscono in 3 bit (**poco interessante**)
 - Trovare due documenti PDF che "collidono" (**più interessante**)
 - Trovare due eseguibili Win32 che collidono (**molto interessante**)
 - Rompere protocolli crittografici in uso (**molto interessante**)

Come funzionano questi attacchi?



- Trovare collisioni per 2 msg che differiscono in soli 3 bit
 - Pochi minuti su un portatile!
- Trovare due eseguibili Win32 che collidono
 - <http://www.win.tue.nl/hashclash/SoftIntCodeSign/>
 - 2 giorni su una Playstation 3 (2009)

PROVA IN ITINERE 18 Novembre

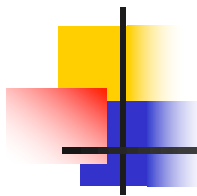


SHA3

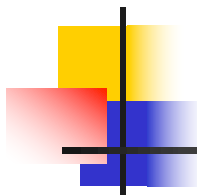
- National Institute for Standards and Technology (NIST) ha ~~terminato~~ concluso la competizione per determinare il nuovo standard

<http://csrc.nist.gov/groups/ST/hash/index.html>

SHA3 - Caratteristiche richieste



- Design
 - Family of functions with 224, 256, 384, 512 bit output sizes
- Compatibilità
 - existing cryptographic standards
- Sicurezza
 - CR, one-wayness, near-collision resistance,...
- Efficienza:
 - as fast as (or faster than) SHA-256

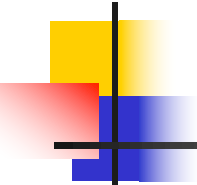


SHA3 Competition

- Submissions: 64
- Round 1: 51 Round 2: 14
- The round 2 functions: BLAKE, Blue Midnight Wish, CubeHash, ECHO, Fugue, Grostl, Hamsi, JH, Keccak, Luffa, Shabal, SHAvite-3, SIMD, Skein.
- Winner: Keccak

EUROPA DOMINA

http://ehash.iaik.tugraz.at/wiki/The_SHA-3_Zoo



La trasformazione Merkle-Damgard

DA FARE

- L'idea base di SHF1 (subroutine di SHA1) e' di iterare l'applicazione della funzione di compressione shf1
- Il metodo e' scelto in modo accurato: se shf1 e' resistente alle collisioni, lo e' anche SHF1.
- Il paradigma mostra come trasformare una funzione di compressione in una funzione hash, in modo che se la prima e' CR, lo e' pure la seconda.
- Adesso guarderemo questo paradigma.

A PARTIRE DA UNA FUNZ. DI COMPRESSIONE
CREIAMO UNA FUNZ. HASH.

SE LA COMPRESSIONE È COLLISION RESISTANT
ALLORA LO È ANCHE LA FUNZIONE HASH



$h : \mathcal{K} \times \{0, 1\}^{b+v} \rightarrow \{0, 1\}^v$ funz. di compr.
resistente alle collisioni

- b lunghezza del blocco, v parametro di concatenazione
- B insieme di stringhe w ($|w|$ multiplo di b), D sottoinsieme di $\{0, 1\}^{<2^b}$

Definizione

$\text{pad} : D \rightarrow B$ e' MD compatibile se $\forall M, M_1, M_2 \in D$

(1) M e' prefisso di $\text{pad}(M)$

(2) Se $|M_1| = |M_2|$, $|\text{pad}(M_1)| = |\text{pad}(M_2)|$

(3) Se $|M_1| \neq |M_2|$, UB di $\text{pad}(M_1) \neq \text{UB}$ di $\text{pad}(M_2)$

- UB: Ultimo blocco *ULTIMO BLOCCO*

SHA È ESATTAMENTE LA TRASFORM. TRAMITE che per SHF 1

SHA1(M) // $|M| < 2^{64}$ *non è l'unica cosa che non va*

$V \leftarrow \mathbf{SHF1}(5A827999 || 6ED9EBA1 || 8F1BBCDC || CA62C1D6, M)$

return V

SHF1(K, M) // $|K| = 128$ e $|M| < 2^{64}$

$y \leftarrow \mathbf{shapad}(M)$

Sia $y = M_1 || M_2 || \dots || M_n$ ($|M_i| = 512$)

$V \leftarrow 67452301 || EFCDA8B9 || 10325476 || C3D2E1F0$

for $i = 1, \dots, n$

$V \leftarrow \mathbf{shf1}(K, M_i || V)$

return V

shapad(M) // $|M| < 2^{64}$

$d \leftarrow (447 - |M|) \bmod 512$ *← LUNGHEZZA UGUALE → d uguale*

Sia ℓ la rappr. binaria (64 bit) di $|M|$

$y \leftarrow M || 1 || 0^d || \ell$ // $|y|$ multiplo di 512

return y

$$y_1 = m_1 || 1 || 0^d || \ell$$

$$y_2 = m_2 || 1 || 0^d || \ell$$

Funzioni hash da funzioni di compressione

$H(K, M)$ $\rightarrow$ ESATTAMENTE SHA 1

$y \leftarrow \text{pad}(M)$

Sia $y = M_1 || M_2 || \dots || M_n$ $\leftarrow$ MULTIPLO DELLA
UN BUCCA
// $(|M_i| = b)$

$V \leftarrow \text{IV}$

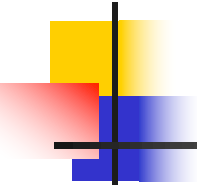
// IV vettore iniziale di v -bit

for $i = 1, \dots, n$

$V \leftarrow h(K, M_i || V)$

Return V

$\leftarrow$ SUPPONIAMO DI AVERE UNA CHIAVE
MA È STANDARD
COMPRESSIQUE



Teorema

- $h: K \times \{0,1\}^{b+v} \rightarrow \{0,1\}^v$ fam. di funz.
- $H: K \times D \rightarrow \{0,1\}^v$ costruita come descritto.
- Sia A_H avversario che trova collisioni in H
- Possiamo costruire A_h che trova collisioni per h e

$$\text{Adv}_H^{\text{cr2-kk}}(A_H) \leq \text{Adv}_h^{\text{cr2-kk}}(A_h)$$

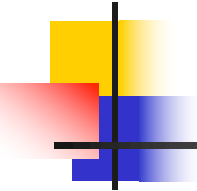
- Il teorema ci dice che se h è resistente alle collisioni, altrettanto deve valere per H

04/11/2025

Sponge Construction

- Paradigma alternativo a Merkle Damgaard
- Semplice processo iterativo basato su una permutazione (senza chiave)
 - $P: \{0,1\}^n \rightarrow \{0,1\}^n$

↑
È SEMPRE QUELLA



Altre differenze

- Positive

- Flexibility – può essere usata anche in modo sicuro anche in contesti nei quali CR non è l'unica proprietà richiesta.

- Negative:

- Collision resistance non può essere facilmente ridotta ad una specifica proprietà di P  PERMUTAZIONE

2 FASI: - ASSORBIMENTO
- SPREMITURA

The sponge construction

FLESSIBILITÀ

- $P: \{0,1\}^n \rightarrow \{0,1\}^n$
- Parametri $r, c : n=r+c$
 - r *rate* (r grande $\Rightarrow$ maggiore efficienza) $\rightarrow$ ITERAZIONI DEL CICLO
 - c *capacity* (c grande $\Rightarrow$ maggiore sicurezza) $\rightarrow$ ITERO PIÙ VOLTE
 - v dimensione dell'output
 - L max dimensione dei messaggi: $\{0,1\}^{<L+1}$
- Il primo passo è il padding $\rightarrow$ LA FORMATTAZIONE DEVE ESSERE INIETTIVA
 - M viene formattato in modo che $|M|$ multiplo di r
 - Nessun requisito particolare (basta che il padding sia iniettivo) ex. 10^*1

The sponge construction

Input: $M \in \{0, 1\}^{\leq L}$, v dimensione dell'output

Output: $h \in \{0, 1\}^v$

// Absorbing stage

$m \leftarrow \text{Pad}(M)$

Sia $m = m_1 || \dots || m_s$ ($|m_i| = r$) $\rightarrow$ scomponiamo

$h \leftarrow 0^n$ // NULL

for $i = 1$ to s do

$m'_i \leftarrow m_i || 0^c \in \{0, 1\}^n$ $\rightarrow$ aggiungo c zero

$h \leftarrow P(h \oplus m'_i)$

// Squeezing stage

$z_1 \leftarrow$ Primi r bit di h $\rightarrow$ primo calcolo

for $i = 2$ to $\lceil v/r \rceil$ do $\rightarrow$ prendiamo r bit alla volta $\rightarrow \lceil v/r \rceil$

$h \leftarrow P(h)$

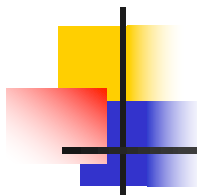
$z_i \leftarrow$ Primi r bit di h

output i primi v bit di $(z_1 || z_2 || \dots || z_{\lceil v/r \rceil})$

Tutto dipende dall'efficienza di P

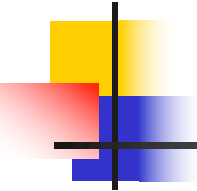
Primo $\rightarrow$
2 perche' il 1
e' gia' calcolato

$\lceil \frac{v}{r} \rceil \cdot r$



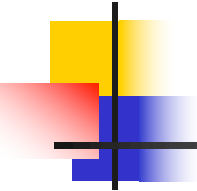
Teorema

- Sia H la funzione hash ottenuta come appena visto (con $v < r+1$)
- Supponiamo che P sia una permutazione random e che 2^c e 2^r siano superpolinomiali
- Allora H è collision resistant



SHA3

- Lo standard NIST specifica una famiglia di funzioni sponge-based
 - Basate sulla permutazione Keccak ^μ
 - Keccak prende in input stringhe di 1600 bit
- Keccak[c] sponge ottenuta da Keccak con
 - Capacità c
 - Usando la 10*1 padding rule
- Keccak[c](m,v) prende input di lunghezza arbitraria e restituisce output di lunghezza v



SHA3

- $\text{SHA3-224}(m) = \text{Keccak}[448](m || 01, 224)$
 - $\text{SHA3-256}(m) = \text{Keccak}[512](m || 01, 256)$
 - $\text{SHA3-384}(m) = \text{Keccak}[768](m || 01, 384)$
 - $\text{SHA3-512}(m) = \text{Keccak}[1024](m || 01, 512)$
-
- Si notino gli extra padding bits!
 - La capacità è sempre doppia dell'output size v
 - Al crescere di v aumenta la sicurezza ma peggiorano le prestazioni